DSA0309 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 1

Morphology-Aware Text Normalisation and Search Layer
for a Hospital Record System

Name: Yoga Madha A

Register Number:192425058

Department / Section:B.Tech AI&ML(3rd Year)

GitHub Link: https://github.com/YOGAMADHA/DSA0309_NLP_Assignment-1

Assumptions and proposed pilot corpus

A reproducible synthetic corpus is proposed because the assignment permits real or synthetic de-identified records. The pilot design contains 500 synthetic records: 150 discharge summaries, 125 prescriptions, 125 operation notes and 100 nursing observations. The target size is at least 25,000 word tokens. The examples below use fictional identifiers and values only.

Record type	Records	Example content
Discharge summary	150	Diagnosis, admission/discharge dates, procedures and outcome
Prescription	125	Drug, strength, dose, route and frequency
Operation note	125	Procedure, anatomy, findings and post-operative plan
Nursing observation	100	Vitals, laboratory values, ward/bed and symptom duration
Total	500	Target token count: 25,000 or more

Evaluation plan: create 40 clinician-style queries with binary relevance judgements and hand-annotate 200 word forms with lemma/stem and grammatical features. All performance figures in this document that are not directly calculable from a specified example are demonstration values and must be rerun on the completed corpus before submission as experimental findings.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis

1.1 Levels of linguistic analysis

Level	Meaning	Hospital-record example	Requirement
Phonology	Sound systems and pronunciation.	A spoken form of a drug name may be transcribed differently.	Not required for typed text; becomes relevant only if speech input is added.
Morphology	Word structure: roots, stems and affixes.	fractures → fracture + noun + plural; nephrectomies → nephrectomy + plural.	Essential.
Syntax	How words combine into phrases and sentences.	"No evidence of infection" differs from "evidence of infection".	Useful for negation and relations; basic indexing can begin without full parsing.
Semantics	Meaning of words and expressions.	fracture and fractured are related; renal and nephro- concern the kidney.	Important for clinically safe concept search.
Pragmatics	Meaning depending on	"Rule out pneumonia"	Needed for clinical

	speaker, purpose and context.	is not a confirmed diagnosis.	interpretation; not solved by regex or stemming.
Discourse	Meaning across sentences and document structure.	A discharge summary may refer back to a diagnosis stated earlier.	Useful for longitudinal interpretation; not essential to the initial finite-state layer.

The first implementation must focus on morphology and structured-entity recognition. Tokenisation is a prerequisite. Syntax, semantics, pragmatics and discourse become important when search results must distinguish negated, suspected, historical and confirmed conditions.

1.2 Three model families

Family	**Best hospital task**	**Why suitable**	**Why alternatives are weaker**
State machines	Validate patient IDs, dates and structured formats.	A finite set of states and transitions recognises exact character sequences efficiently.	A probabilistic model is unnecessary for strict formats; a hand-written rule alone does not naturally show state transitions.
Formal rule systems	Expand abbreviations and apply spelling/morphology rules.	Rules are explicit, auditable and reproducible.	A state machine alone does not conveniently express all lexical replacement policies; probabilities may make deterministic normalisation unpredictable.
Probabilistic models	Rank ambiguous search results or identify noisy OCR tokens.	They can use context and learned likelihoods.	A deterministic regex is too rigid for uncertain OCR; manually written rules cannot cover every context.

1.3 Ambiguity in the corpus

- Lexical ambiguity: “lead” may refer to a metal or the verb “to lead”; the clinical context determines the intended sense.
- Morphological ambiguity: “findings” is normally find + noun plural, but “findings” can also be a lexicalised noun in medical reporting.
- Segmentation ambiguity: OCR may produce “bloodpressure” or split a word across lines as “dis-charge”.
- Abbreviation ambiguity: “MS” may mean multiple sclerosis, mitral stenosis or morphine sulphate depending on context.

- Additional example – scope/negation ambiguity: “no fracture seen” must not be indexed as a confirmed fracture without preserving negation.

1.4 Search and resource use

Automata use state-space traversal: processing a string of length n takes O(n) time for a deterministic finite automaton and typically O(number of active states × n) for an NFA simulation. A DFA may require more memory because subset construction can create up to 2^n states in the worst case. Regex engines may use backtracking; poorly designed expressions with nested ambiguous quantifiers can take excessive time. A finite-state pipeline needs a lexicon and rules but does not require labelled training data. Building and validating the lexicon, abbreviations and clinical patterns consumes expert effort.

1.5 Proposed pipeline

Clinical records → [1. Input cleaning / OCR repair: rules + optional weighted edit distance] → [2. Tokenisation: regex + maximal munch] → [3. Entity recognition: regex + DFA/NFA validators] → [4. Case/abbreviation/date/unit normalisation: rewrite rules] → [5. Morphological analysis: lexicon FST + spelling rules] → [6. Search indexing: lemma/features + Porter fallback] → [7. Query matching and ranking: inverted index + optional probabilistic ranking] → Search results

Input: de-identified free-text hospital records and clinician queries. Output: normalised tokens, recognised structured entities, morphological analyses and searchable index entries.

TASK 2 – Basic Regular Expression Patterns

2.1 Basic operators

Operator	Meaning	Clinical example
Concatenation	Items occur in sequence.	AB[0-9]{6}
Disjunction |	Either alternative is accepted.	mg|mcg
Kleene star *	Zero or more repetitions.	[0-9]*
Plus +	One or more repetitions.	[0-9]+
Optional ?	Zero or one occurrence.	(kg)?
Character class	One character from a set.	[A-Z]
Range	Characters within a range.	[0-9]
Negated class	Any character except listed characters.	[^\n]+
Anchors ^ and $	Beginning and end of string.	^AB[0-9]{6}$
Counters {m,n}	A bounded number of repetitions.	[0-9]{2,4}

2.2 Twelve entity patterns

Entity	Regex (Python raw-string form)	Must match	Must not match
Patient identifier	\\b(?:PT|MRN)[-]?[0-9]{6,10}\\b	PT-1234567	PT-12A4567; PT-123
Admission / discharge date	\\b(?:[0-3]?[0-9][/-][01]?[0-9][/-](?:19|20)?[0-9]{2}|(?:19|20)[0-9]{2}-[01][0-9]-[0-3][0-9])\\b	05/11/2026; 2026-11-05	39/19/2026

Drug + strength	\\b[A-Za-z][A-Za-z-]*(?:\\s+[A-Za-z][A-Za-z-]*)?\\s+[0-9]+(?:\\.[0-9]+)?\\s?(?:mg|mcg|g|mL)\\b	paracetamol 500mg	paracetamol five mg
Dosage frequency	\\b(?:OD|BD|TDS|QID|BID|TID|Q[0-9]{1,2}H|once daily|twice daily)\\b	BD; every 8 hours (extend pattern)	dailyish
Lab value + unit	\\b[A-Za-z][A-Za-z0-9-]*\\s*[:=]?\\s*[0-9]+(?:\\.[0-9]+)?\\s?(?:mg/dL|g/dL|mmol/L|IU/L|10\\^?[0-9]+/L)\\b	Hb 12.5 g/dL	Hb twelve g/dL
Blood pressure	\\b(?:BP\\s*)?[0-9]{2,3}\\s*/\\s*[0-9]{2,3}\\s?mmHg\\b	BP 120/80 mmHg	BP 120/800 mmHg (range validation needed)
Temperature	\\b(?:Temp\\s*)?(?:3[0-9]|4[0-2])(?:\\.[0-9])?\\s?°?[CF]\\b	Temp 37.2 C	Temp 99.9 C (physiological range check)
ICD code	\\b[A-Z][0-9][A-Z0-9](?:\\.[A-Z0-9]{1,4})?\\b	S72.001A	72S.00
Ward and bed	\\bWard\\s*[A-Z0-9-]+\\s*,?\\s*Bed\\s*[A-Z0-9-]+\\b	Ward B2, Bed 14	Ward, Bed
Contact number	\\b(?:\\+91[-]?)?[6-9][0-9]{9}\\b	9876543210	+91 12345
Age	\\b(?:Age\\s*[:=]?\\s*)?[0-9]{1,3}\\s?(?:years?|yrs?|y/o)\\b	Age: 45 years	Age 450 years
Symptom duration	\\b[0-9]+\\s?(?:hours?|days?|weeks?|months?|years?)\\b	3 days	many days

These expressions are candidate recognisers, not complete clinical validators. Add semantic range checks after regex matching: valid calendar dates, plausible age and temperature, valid blood-pressure ranges, known drug names and locally approved identifier formats. The assignment's inventory is therefore covered while invalid-but-syntactically-similar strings are rejected by a validation stage.

2.3 Greedy and non-greedy matching

Text: "Give 500mg paracetamol and 250mg amoxicillin."

Greedy: r'Give.*mg' → 'Give 500mg paracetamol and 250mg'

Lazy: r'Give.*?mg' → 'Give 500mg'

The greedy quantifier .* consumes as much text as possible while allowing the expression to succeed, so it reaches the last “mg”. The lazy quantifier .*? consumes the shortest span that allows success, so it stops at the first “mg”. For clinical extraction, bounded patterns around a drug name and strength are safer than a broad dot-star.

2.4 Precedence and grouping

Intended: ^(AB|CD)[0-9]{4}$

Misplaced: ^AB|CD[0-9]{4}$

The intended expression accepts either AB followed by four digits or CD followed by four digits. Without grouping, alternation has lower precedence than concatenation, so the second expression means “starts with AB” OR “ends with CD followed by four digits”; it can accept unintended strings such as ABXYZ.

2.5 Evaluation and error analysis

Because the actual corpus was not supplied, the following is a clearly labelled pilot reporting format, not a measured result. After annotation, calculate precision = TP/(TP+FP) and recall = TP/(TP+FN) for each entity type.

Pattern group	Illustrative TP	FP	FN	Likely error cause
Identifiers / contacts	98	2	2	OCR character substitutions; spacing variations
Dates	94	4	6	Invalid dates and locale ambiguity
Drug + strength	90	7	10	Brand names, missing spaces, misspellings
Frequency / duration	92	5	8	Unlisted abbreviations and language variation
Lab / BP / temperature	95	3	5	Unit variants and physiologically implausible values
ICD / ward-bed / age	96	3	4	Local formats and incomplete labels

The figures above are placeholders for demonstration. A final report must show actual TP, FP and FN counts from the 500-record corpus and explain every error using its record identifier and text span.

TASK 3 – Regular Expressions for Tokenisation and Normalisation

3.1 Token definition and tokenizer

A token is a maximal meaningful sequence consisting of a word, a recognised abbreviation, a decimal number, a number-with-unit expression, or a hyphenated clinical term. Punctuation is retained separately when it carries meaning, such as a decimal point or slash in a measurement.

```
TOKEN = r'''(?ix)
    [A-Za-z]+(?:\.[A-Za-z]+)+\.?          # abbreviations such as i.v.
  | [A-Za-z]+(?:-[A-Za-z]+)+              # post-operative
  | [0-9]+(?:\.[0-9]+)?(?:\s?(?:mg|mcg|g|mL|mmHg|kg|cm)) # value + unit
  | [0-9]+(?:/[0-9]+)+                    # BP or ratio
```

```
  | [A-Za-z]+(?:'[A-Za-z]+)?          # words / contractions
  | [0-9]+(?:\.[0-9]+)?             # numbers
'''
```

Apply alternatives in the listed priority order and choose the longest valid match at the current position (maximal munch). In production, unit tokens should be checked against a controlled unit list.

3.2 Maximal munch examples

Input	Whitespace split	Maximal-munch tokens
BP 120/80 mmHg	BP | 120/80 | mmHg	BP | 120/80 | mmHg (measurement preserved as a structured value)
Give 500mg paracetamol	Give | 500mg | paracetamol	Give | 500mg | paracetamol (number and unit kept together)
post-operative review	post-operative | review	post-operative | review (hyphenated word kept as one token)
i.v. fluids started	i.v. | fluids | started	i.v. | fluids | started (abbreviation kept intact)

Whitespace splitting happens to preserve some examples but does not define internal token boundaries. For example, “500 mg” may be split into two tokens unless a contextual number-unit rule joins it.

3.3 Normalisation substitutions

Operation	Example input	Canonical output
Case folding	PARACETAMOL	paracetamol
Abbreviation expansion	BP	blood pressure
Abbreviation expansion	HR	heart rate
Abbreviation expansion	RR	respiratory rate
Abbreviation expansion	Temp	temperature
Abbreviation expansion	SOB	shortness of breath
Abbreviation expansion	N/V	nausea and vomiting
Abbreviation expansion	c/o	complains of
Abbreviation expansion	h/o	history of
Abbreviation expansion	PMH	past medical history
Abbreviation expansion	DM	diabetes mellitus
Abbreviation expansion	HTN	hypertension
Abbreviation expansion	CAD	coronary artery disease
Abbreviation expansion	CVA	cerebrovascular accident
Abbreviation expansion	MI	myocardial infarction
Abbreviation expansion	UTI	urinary tract infection
Abbreviation expansion	URI	upper respiratory infection
Abbreviation expansion	COPD	chronic obstructive pulmonary disease
Abbreviation expansion	CKD	chronic kidney disease
Abbreviation expansion	AKI	acute kidney injury
Abbreviation expansion	CBC	complete blood count

Abbreviation expansion	RFT	renal function test
Abbreviation expansion	LFT	liver function test
Abbreviation expansion	ECG	electrocardiogram
Abbreviation expansion	MRI	magnetic resonance imaging
Abbreviation expansion	CT	computed tomography
Abbreviation expansion	IV	intravenous
Abbreviation expansion	IM	intramuscular
Abbreviation expansion	OD	once daily (only in medication-frequency context)
Abbreviation expansion	BD	twice daily (only in medication-frequency context)
Abbreviation expansion	TDS	three times daily (only in medication-frequency context)
Abbreviation expansion	QID	four times daily (only in medication-frequency context)
Abbreviation expansion	SOS	when required (only in medication context)

Date canonicalisation: 05/11/2026 → 2026-11-05 only when the source locale is known to be day/month/year. Unit canonicalisation: 500 MG → 500 mg; 0.5 g → 500 mg only when conversion is safe and the original value/unit are retained in metadata.

3.4 Full-record before and after

Before: PT-1234567 | 05/11/2026 | c/o fever x 3 days. BP 120/80 mmHg. Rx: PARACETAMOL 500 MG BD.

After: patient_id=PT-1234567 | date=2026-11-05 | complains of fever duration=3 days. blood pressure=120/80 mmHg. prescription: paracetamol 500 mg twice daily.

3.5 Irreversible decisions and limits

- Case folding removes original capitalisation. It is acceptable for case-insensitive search but not when case distinguishes an identifier or gene symbol.
- Abbreviation expansion can lose the exact shorthand used by the clinician. Preserve the original span alongside the expanded form.
- Unit conversion can remove the original unit and precision. Store both original and canonical values.
- A regular expression cannot recognise arbitrarily nested balanced parentheses. This language requires matching an unbounded number of opening and closing brackets; a finite automaton has only finite memory and cannot count arbitrary nesting depth.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence

4.1 Formal definition

A finite automaton is a five-tuple $(Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states, Σ is the input alphabet, δ is the transition function, q_0 is the start state and F is the set of accepting states.

4.2 Four DFA validators

Validator	State path (accepting state marked *)	Transition summary
Patient ID: PT followed by 6 digits	q0 --P--> q1 --T--> q2 --digit--> q3 --digit--> q4 --digit--> q5 --digit--> q6 --digit--> q7 --digit--> q8*	q0:P→q1; q1:T→q2; q2:digit→q3; q3:digit→q4; q4:digit→q5; q5:digit→q6; q6:digit→q7; q7:digit→q8; all other inputs→dead
Date: YYYY-MM-DD (shape validation)	q0 --digit×4--> q4 --' - '--> q5 --digit×2--> q7 --' - '--> q8 --digit×2--> q10*	q0-3:digit→next; q4:'-'→q5; q5-6:digit→next; q7:'-'→q8; q8-9:digit→next; other→dead
Blood pressure: 2–3 digits / 2–3 digits mmHg	q0 --digits--> q1 --/--> q2 --digits--> q3 --space--> q4 --mmHg--> q8*	q0:digit→q1; q1:digit→q1 or '/'→q2; q2:digit→q3; q3:digit→q3 or space→q4; q4:m→q5; q5:m→q6; q6:H→q7; q7:g→q8; other→dead
ICD-like code: letter + digit + alphanumeric + optional decimal	q0 --letter--> q1 --digit--> q2 --alnum--> q3* --'.'--> q4 --alnum--> q5*	q0:letter→q1; q1:digit→q2; q2:alnum→q3; q3:'.'→q4; q4:alnum→q5; q5:alnum→q5; other→dead

Each validator has at least six operational states when character-position states are expanded. A dead state is included for every unexpected character. The diagrams are represented textually here; each arrow is a labelled transition.

4.3 NFA with epsilon transitions

Use an NFA for a patient identifier with optional separator: PT, optionally followed by '-' or a space, then six digits. After reading PT, the machine can take an ε-transition directly to the first digit state or consume an optional separator.

States: s0(start), s1, s2, s3, s4, s5, s6, s7, s8(accept), dead

s0 -P→ s1; s1 -T→ s2; s2 -ε→ s3; s2 -'-'→ s3; s2 -space→ s3; s3 -digit→ s4 -digit→ s5 -digit→ s6 -digit→ s7 -digit→ s8 -digit→ s9(accept)

Accepted trace for PT-123456: after P {s1}; after T {s2,s3}; after '-' {s3}; after 1 {s4}; after 2 {s5}; after 3 {s6}; after 4 {s7}; after 5 {s8}; after 6 {s9}. Rejected trace for PT-12A456: after A there is no digit transition, so the active-state set becomes empty and the string is rejected.

4.4 Subset construction and minimisation

Subset construction treats each DFA state as a set of NFA states. The ε-closure of a set is computed before and after each transition.

DFA state	NFA subset	Input digit	Input separator	Accepting?
A	{s0}	∅	∅	No
B	{s1}	∅	∅	No
C	{s2,s3}	{s4}	{s3}	No

D	{s4}	{s5}	∅	No
E	{s5}	{s6}	∅	No
F	{s6}	{s7}	∅	No
G	{s7}	{s8}	∅	No
H	{s8}	{s9}	∅	No
I	{s9}	∅	∅	Yes
Ø	∅	Ø	Ø	No

For the simplified illustrative NFA, the reachable DFA has 10 subsets including the dead subset. Minimisation merges no distinct position states because each has a different remaining-length requirement; the resulting DFA therefore has 10 states. For a complete implementation, recompute reachable subsets from the exact NFA transition graph and run partition refinement; the table above is a teaching trace of the construction.

4.5 Thompson construction and reverse conversion

For the expression PT-?[0-9]{6}, Thompson construction creates small NFAs for P, T, optional hyphen and six concatenated digit transitions. Concatenation links the accepting state of one fragment to the start of the next with ε-transitions; optionality adds a bypass ε-transition.

The reverse conversion can be done by state elimination. Add a new start and final state, label transitions with regular expressions, then eliminate intermediate states using R'ij = Rij | Rik(Rkk)*(Rkj). Eliminating the intermediate states of the patient-ID DFA yields PT-?[0-9][0-9][0-9][0-9][0-9][0-9]. The expression and automaton accept the same set: PT-123456 and PT123456 are accepted; PT-12345 and PT-12A456 are rejected.

4.6 Pumping lemma

Pumping lemma: if L is regular, there is a pumping length p such that every string s in L with $|s| \geq p$ can be written s = xyz, where $|xy| \leq p$, $|y| > 0$ and xy^iz is in L for every $i \geq 0$.

Consider L = { $'^n)'^n : n \geq 0$ }, the language of strings with equal numbers of opening and closing brackets in the required order. Assume L is regular with pumping length p. Choose $s = '^p)'^p$. Since $|xy| \leq p$, y contains only opening brackets. Pumping y out (i=0) produces fewer opening brackets than closing brackets, so the result is not in L. This contradicts the lemma; therefore L is not regular. A finite-state regex layer cannot validate arbitrary balanced nesting.

TASK 5 – Inflectional Morphology

A morpheme is the smallest unit carrying grammatical or lexical meaning. A stem is the form to which an affix attaches. An affix is a bound morpheme attached to a stem. Inflection changes grammatical features without normally changing word class or core dictionary meaning. A paradigm is the set of inflected forms of a lexeme.

Inflection differs from derivation because it generally preserves grammatical category, is more productive, and commonly occurs outside derivational material (for example, fracture + s; nephrectomy + s). Derivation can create a new lexeme or change category (operate → operation).

5.1 Inflectional paradigms

5.2 Five clinical lexeme paradigms

Lexeme	Plural / 3rd person	Past	-ing / comparison
fracture	fractures	fractured	fracturing
lesion	lesions	—	—
treat	treats	treated	treating
bleed	bleeds	bled	bleeding
severe	—	—	more severe / most severe

5.3 English spelling rules

Rule	Implementation condition	Example	Over-generation counter-example
e-deletion	Drop final silent e before -ing; retain before most other suffixes.	treat + ing → treating	see + ing → seeing (do not drop both e letters)
Consonant doubling	Double final consonant in a stressed CVC base before vowel-initial suffix where applicable.	admit + ed → admitted	visit + ed → visited (final stress not on last syllable)
y-to-i	Consonant + y changes y to i before -es/-ed; not before -ing.	biopsy + es → biopsies	play + ed → played (vowel before y)
sibilant -es	Add -es after s, x, z, ch, sh.	rash + es → rashes	stomach + s → stomachs (ch sound/spelling exceptions require lexical checking)

5.4 Irregular forms

Base	Irregular form	Reason
be	am/is/are/was/were/been	Suppletion; form cannot be generated by a regular suffix rule.
go	went/gone	Suppletive past form.
have	has/had	Irregular agreement and past.
do	does/did/done	Irregular spelling and stem change.
bleed	bled	Vowel change and suffix pattern.
diagnose	diagnosed	Regular spelling but keep lexical record for reliable analysis if exceptions are encoded.

child	children	Irregular plural.

The system stores irregular mappings in the lexicon so that analysis is explicit and reversible.

5.5 Type-to-lemma ratio and retrieval effect

Type-to-lemma ratio = number of distinct surface word types / number of distinct lemmas. Example pilot calculation: 4,200 surface types / 3,000 lemmas = 1.40. If 40 queries contain 100 relevant missed matches and 28 are recovered by inflectional conflation alone, the inflection-only share is 28%. These are illustrative values; the final values must be calculated from the annotated corpus and query judgements.

TASK 6 – Derivational Morphology

Derivation attaches an affix to a base to form a new lexeme or alter its meaning or grammatical category. It is less predictable than inflection because affixes are selective, meanings can be specialised and derived words may be lexicalised.

6.1 Derivational affix inventory

Affix	Example	Typical effect
-itis	arthritis	noun: inflammation condition
-ectomy	nephrectomy	noun: surgical removal
-ology	cardiology	noun: field/study
-osis	fibrosis	noun: condition/process
-aemia	anaemia	noun: blood condition
-ation	amputation	noun: process/result
-able	treatable	adjective: capable of being treated
-ive	obstructive	adjective: having a quality/tendency
-al	renal	adjective: relating to
-ic	cardiac	adjective: relating to
-ist	neurologist	noun: specialist/person
-er	transporter	noun: agent/device
nephro-	nephrology	combining form: kidney
hyper-	hypertension	prefix: excessive/high
hypo-	hypoglycaemia	prefix: below/deficient
re-	reoperation	prefix: again
un-	unhealed	prefix: not/reversal

6.2 Derivational families and affix order

Root/family	Derivational family	Order / observation
nephr-	nephro + logy → nephrology; nephro + ectomy → nephrectomy; nephrectomy + s → nephrectomies	Root/combining form + derivational suffix; plural inflection attaches last.
cardi-	cardio + logy → cardiology;	Combining form + suffix.

	cardi + ac → cardiac	
operat-	operate → operation → operational	Verb → noun → adjective.
infect-	infect → infection → infectious	Verb → noun → adjective.
treat-	treat → treatable; re + treat → retreat/re-treat (context-dependent)	Suffix and prefix attach to base; meaning depends on context.
path-	pathology; pathological	Field noun → adjective.

Impossible or inappropriate orders include *-ectomy + nephro- where the intended English clinical compound is not formed by simply reversing nephrectomy; and *-s + -ectomy, because plural inflection does not normally precede derivational suffixation. The lexicon should record attested forms rather than generate every theoretical combination.

6.3 Semantic non-compositionality

- "Inflammation" is not always simply a transparent sum of the root and -ation; its clinical sense is conventionalised.
- "Nephrectomy" denotes surgical removal of a kidney, not merely any event involving nephro- and -ectomy in an unrestricted sense.
- "Anaemia" has a specific clinical meaning and should not be treated as any generic "lack of blood" phrase.

Aggressive stemming can merge words that share letter sequences but have different clinical concepts. A safe system retains the original token, lemma, affix analysis and concept identifier separately.

6.4 Productivity and search conflation

Productivity should be measured by counting distinct derived types and total tokens for each affix. A high number of distinct types and new formations indicates productivity; a small closed set of conventional words suggests fossilisation. Without the corpus, counts cannot be reported as observations.

Affix	**Illustrative type count**	**Illustrative token count**	**Provisional classification**
-ation	22	145	Productive
-al	18	120	Productive
-itis	12	74	Moderately productive
-ectomy	8	39	Specialised/productive within procedures
-ology	6	21	Limited
-aemia	4	17	Mostly specialised/fossilised
hyper-	9	48	Productive
hypo-	8	41	Productive

Conflation that helps: fracture/fractures/fractured; nephrectomy/nephrectomies. Potentially misleading conflation: hyperglycaemia with hypoglycaemia; nephritis with nephrectomy. Search should conflate inflectional forms by default, but derivational relatives should be linked as related concepts with ranking and context controls rather than collapsed into one stem.

TASK 7 – Finite-State Morphological Parsing

A finite-state transducer (FST) is a two-tape automaton. One tape contains lexical symbols (lemma and grammatical features); the other contains surface symbols (the written word). A transducer maps between the tapes and can be run from surface to lexical analysis or from lexical input to surface generation.

7.1 Example and continuation classes

Surface tape: fractures ⇄ Lexical tape: fracture +N +PL

The lexicon is organised into continuation classes: Noun entries continue to noun inflectional endings; Verb entries continue to verb endings; Adjective entries continue to comparative/superlative endings. Derivational continuation classes connect roots to derivational suffixes, and the resulting lexeme may then enter an inflectional class.

Class	Lexical entries	Allowed continuations
NOUN	fracture, lesion, kidney, nephrectomy	+SG (zero), +PL (-s/-es/-ies)
VERB	treat, admit, bleed, heal	+3SG (-s/-es), +PAST (-ed/irregular), +PROG (-ing)
ADJECTIVE	mild, severe, high	+COMP (-er/more), +SUPER (-est/most)
DERIVATION	nephr-, infect-, operat-, cardi-	-itis, -ectomy, -ology, -ation, -al, -ic

7.2 Hand-traced analyses

Surface form	Symbols consumed on surface tape	Lexical output	Analysis
fractures	f r a c t u r e s	fracture +N +PL	Plural noun
treated	t r e a t e d	treat +V +PAST	Past-tense verb
bleeding	b l e e d i n g	bleed +V +PROG	Progressive verb
nephrectomies	n e p h r e c t o m i e s	nephrectomy +N +PL	Plural procedure noun
treatable	t r e a t a b l e	treat +V +ABLE	Derived adjective

The state sequence follows the lexicon path to the stem, enters the appropriate continuation class, and consumes/emits the suffix mapping. For fractures, the lexical stem fracture is emitted while the surface suffix s is mapped to +PL. For nephrectomies, the spelling alternation y → ies is handled by the orthographic rules in Task 8.

7.3 Generation with the same transducer

Lexical input	Generated surface form

fracture +N +PL	fractures
treat +V +PAST	treated
bleed +V +PROG	bleeding
nephrectomy +N +PL	nephrectomies
treat +V +ABLE	treatable

The same relation is traversed in reverse: parsing maps surface to lexical analysis, while generation maps lexical-plus-feature input to surface spelling. Separate machines are not required; the direction of traversal changes.

7.4 Ambiguity

- "treated" may be analysed as treat +V +PAST or as an adjective-like participial form treat +V +PASTPART, depending on the grammar.
- "healing" may be heal +V +PROG or a noun/adjective use, heal +N/ADJ +DERIVED, depending on sentence context.

The FST returns all licensed analyses. A downstream tagger or clinical context model selects the most likely interpretation using neighbouring words, sentence role and document type. The original surface form is retained.

TASK 8 – Orthographic Rules and Two-Level Morphology

A lexicon transducer alone cannot simply concatenate every stem and suffix because English spelling changes at morpheme boundaries. For example, biopsy + s should produce biopsies, not *biopsys; admit + ed produces admitted, not *admited; and treat + ing produces treating, not *treateing.

8.1 Rewrite rules

Rule	**Context and example**
e → 0 / C __ +ing	Delete final silent e before -ing: treat+ing → treating.
y → i / C __ +es	Change consonant+y before plural -es: biopsy+s → biopsies.
y → i / C __ +ed	Change consonant+y before -ed: biopsy+ed → biopsied (where verb use is valid).
0 → C / stressed CVC __ vowel suffix	Double final consonant where stress and orthographic conditions allow: admit+ed → admitted.
+es after sibilant	Add e before plural s after s, x, z, ch, sh: rash+s → rashes.
Keep y / V __ +s	Do not change y after a vowel: artery? (not this rule); play+s → plays.

8.2 Cascade and order

Lexicon FST → morpheme boundary rules (y-to-i / e-deletion / consonant doubling) → suffix realisation → surface form

Rule ordering matters. The boundary must be identified before spelling changes are applied. The y-to-i rule must see the suffix boundary; e-deletion must apply only before the appropriate suffix; consonant doubling requires syllable/stress information or a lexicon flag. A practical cascade uses lexicon flags for exceptions and composes the rule transducers with the lexical FST.

8.3 Sequential cascade vs two-level morphology

Approach	Description	Advantage	Limitation
Sequential cascade	Apply rule transducers one after another.	Easy to inspect, debug and order.	Earlier rules can change the input seen by later rules; ordering errors are possible.
Two-level (Koskenniemi)	Lexical and surface symbols are related in parallel by constraints at the same time.	Spelling alternations are stated as local correspondences; avoids some cascade ordering problems.	More complex rule notation and implementation; constraints can be harder for beginners to debug.

8.4 Test set

Lexical input	Expected surface	Result
treat + ing	treating	Pass
admit + ed	admitted	Pass
biopsy + plural	biopsies	Pass
rash + plural	rashes	Pass
play + 3SG	plays	Pass
fracture + plural	fractures	Pass
lesion + plural	lesions	Pass
study + past	studied	Pass
stop + ing	stopping	Pass
make + ing	making	Pass
bleed + past	bled	Lexicon irregular
go + past	went	Lexicon irregular
have + 3SG	has	Lexicon irregular
see + ing	seeing	Exception / retain e
visit + past	visited	No doubling
open + past	opened	No doubling
fix + plural	fixes	Pass
box + plural	boxes	Pass
kidney + plural	kidneys	Vowel+y retains y
nephrectomy + plural	nephrectomies	Pass

Illustrative test outcome: 17 rule/lexicon cases pass directly and 3 require exception handling or lexical constraints. In a final implementation, record the actual output string and failure cause for every test rather than relying only on a pass label.

TASK 9 – The Porter Stemmer

9.1 Definitions

Porter stemming is a suffix-stripping algorithm, not a dictionary-based lemmatiser. A consonant is a letter that is not a vowel (with y treated contextually); a vowel is a, e, i, o or u, with y acting as a vowel when preceded by a consonant. The measure m counts vowel-consonant sequences in the stem, conventionally represented as [C](VC)^m[V].

Conditions: *v* means the stem contains a vowel; *d means the stem ends in a double consonant; *o means the stem ends in consonant-vowel-consonant where the final consonant is not w, x or y.

9.2 Worked measure examples

Word	Approximate C/V pattern	m	Observation
fracture	CCVCCVCV	2	Two VC transitions.
treated	CCVVCVC	2	Suffix removal exposes stem structure.
lesions	CVVCVCC	2	Plural s is removable.
bleeding	CCVVCVVC	2	-ing removal leaves bleed.
kidneys	CVCCVVC	2	Plural s can be removed.
infection	VCCVCCVC	2	Derivational ending may be reduced in later steps.

9.3 Five algorithm steps

The original Porter algorithm is conventionally described as five steps of ordered suffix transformations. The exact rule applied depends on m and on the surrounding stem pattern. The following traces are illustrative English Porter-style traces; clinical terms may need a domain lexicon to prevent harmful conflation.

Input	Step 1	Step 2	Step 3	Step 4	Step 5 / final stem
fractures	fracture (remove s)	fracture	fracture	fracture	fractur
treated	treat (remove ed)	treat	treat	treat	treat
lesions	lesion (remove s)	lesion	lesion	lesion	lesion
bleeding	bleed (remove ing)	bleed	bleed	bleed	bleed
kidneys	kidney (remove s)	kidney	kidney	kidney	kidney
infection	infection	infect	infect	infect	infect

The final form “fractur” illustrates why a stem is not necessarily a valid dictionary word or lemma. In practice, use NLTK's PorterStemmer for the exact standard implementation and compare its output with the finite-state lemmatiser.

9.4 Stemming vs lemmatisation

Surface	Porter-like stem	FST lemma	Agreement
fractures	fractur	fracture	No
treated	treat	treat	Yes
lesions	lesion	lesion	Yes
bleeding	bleed	bleed	Yes
nephrectomies	nephrectomi	nephrectomy	No
went	went	go	No

For the required 200 annotated forms, calculate agreement = matching stem/lemma outputs ÷ 200. Illustrative pilot: 148/200 = 74% agreement; divergence is expected for irregulars and orthographic endings.

9.5 Error types and retrieval

- Over-stemming: two unrelated clinical terms are reduced to the same stem, potentially merging distinct concepts. Example risk: “organ” and “organisation” may be conflated by an aggressive suffix rule.
- Under-stemming: related forms remain separate. Example: nephrectomies may not reduce to the same canonical lemma as nephrectomy.

Illustrative annotated-set rates: 8% over-stemming and 11% under-stemming. These are placeholders and must be replaced by manual error labels on the 200 forms.

Retrieval condition	Precision (illustrative)	Recall (illustrative)	F1
No stemming	0.91	0.68	0.78
Porter stemmer	0.84	0.82	0.83
Full FST analysis	0.90	0.91	0.90

F1 = 2 × precision × recall / (precision + recall). Full morphological analysis is expected to improve recall while retaining better precision than broad suffix stripping, provided the lexicon and ambiguity handling are maintained.

TASK 10 – Implement and Test the Pipeline

10.1 Python implementation outline

```
import re
from nltk.stem import PorterStemmer

TOKEN_RE = re.compile(
```

```
    r"(?ix)[A-Za-z]+(?:\.[A-Za-z]+)+\.?|"
    r"[A-Za-z]+(?:-[A-Za-z]+)+|"
    r"\d+(?:\.\d+)?\s?(?:mg|mcg|g|mL|mmHg|kg|cm)|"
    r"\d+(?:/\d+)+|[A-Za-z]+(?:'[A-Za-z]+)?|\d+(?:\.\d+)?"
)

PATTERNS = {
    "patient_id": re.compile(r"\b(?:PT|MRN)[- ]?\d{6,10}\b"),
    "date": re.compile(r"\b(?:\d{4}-\d{2}-\d{2}|\d{1,2}/\d{1,2}/\d{2,4})\b"),
    "drug_strength": re.compile(r"\b[A-Za-z][A-Za-z-]*\s+\d+(?:\.\d+)?\s?(?:mg|mcg|g|mL)\b", re.I),
    "bp": re.compile(r"\b\d{2,3}\s*/\s*\d{2,3}\s?mmHg\b", re.I),
}

ABBREVIATIONS = {
    "bp": "blood pressure", "hr": "heart rate", "rr": "respiratory rate",
    "sob": "shortness of breath", "htn": "hypertension",
    "dm": "diabetes mellitus", "iv": "intravenous",
    "od": "once daily", "bd": "twice daily", "tds": "three times daily"
}

def tokenize(text):
    return TOKEN_RE.findall(text)

def normalise(text):
    text = text.lower()
    for short, full in ABBREVIATIONS.items():
        text = re.sub(rf"\b{re.escape(short)}\b", full, text)
    return text

def recognise(text):
    return {name: pattern.findall(text) for name, pattern in PATTERNS.items()}

stemmer = PorterStemmer()
def porter_stem(word):
    return stemmer.stem(word)

LEXICON = {"fractures": [("fracture", "+N", "+PL")],
           "treated": [("treat", "+V", "+PAST")],
           "bleeding": [("bleed", "+V", "+PROG")]}
def analyse(word):
    return LEXICON.get(word.lower(), [])
```

This standard-library-plus-NLTK skeleton implements tokenisation, selected entity patterns, abbreviation normalisation, a small FST-style lookup demonstration and Porter stemming. Extend the lexicon to at least 300 stems, affixes to the required counts, and implement explicit automaton transition tables/subset construction for the full submission.

10.2 Test plan

- For each DFA, test at least one valid and one invalid string, including malformed length and invalid characters.
- For every regex pattern, keep positive and negative test strings and compare the matched span with the expected span.
- Test ambiguous FST forms and verify that all licensed analyses are returned.
- Test irregular forms such as went, was, has and bled.
- Compare program output against the hand traces in Tasks 4, 7 and 9.

10.3 Pipeline results

Stage	Items processed (pilot plan)	Expected output	Rejections / errors
Input cleaning	500 records	500 cleaned records	OCR/encoding exceptions
Tokenisation	≥25,000 tokens	Token spans and punctuation	Unrecognised or malformed spans
Entity recognition	All record strings	12 entity classes	False matches and missed entities
Normalisation	All tokens	Canonical case, abbreviation, date/unit forms	Ambiguous abbreviations
FST analysis	200 annotated forms + corpus tokens	Lemma and feature analyses	Out-of-lexicon tokens
Porter fallback	Unanalysed alphabetic tokens	Fallback stems	Over/under-stemming

Runtime must be measured on the target Windows 11/VS Code environment using time.perf_counter(), with corpus size and hardware recorded. Do not invent runtime values: report measured seconds for each stage after executing the program.

Suggested plot: x-axis = lexicon size (50, 100, 200, 300 stems); y-axis = percentage of annotated tokens receiving an FST analysis. A second useful plot is record length against processing time. These plots should be generated from the actual test log.

TASK 11 – Compare, Recommend and Defend the Normalisation Design

Design	Retrieval quality	Coverage	Runtime	Maintenance
No normalisation	Low recall for variants; high precision for exact matches	High for exact surface forms	Fast	Low

Porter stemming	Recall improves; precision may fall through over-stemming	Broad for unseen English words	Very fast	Low
Full FST analysis	Best balance when lexicon covers domain vocabulary	Limited by lexicon and rules	Linear and predictable	High initial and ongoing expert effort

Recommendation: deploy a hybrid finite-state design. Use regex/DFA validation for structured entities, conservative normalisation for abbreviations and units, an FST for inflection and known derivation, and Porter stemming only as a low-confidence fallback for out-of-lexicon English tokens. Preserve original text and features so that a clinician can inspect the evidence.

Expected failure modes include unseen drug names, ambiguous abbreviations, misspellings, OCR substitutions and non-English morphology. A clinician may notice a relevant record missing from search or an irrelevant result ranked too highly. The interface should display the original matched phrase and the normalised form.

Manual effort includes pattern design, clinical review, 300+ stem lexicon construction, affix documentation, irregular-form listing and annotation of 200 forms plus 40 queries. A learned approach becomes more economical when the vocabulary and error patterns change faster than experts can maintain rules, provided enough representative labelled data and governance are available.

TASK 12 – DAY-4 DESIGN CHALLENGE: Re-evaluate and Adapt the Design

A. Impact Analysis

The new inputs contain OCR character noise, line-break hyphenation, lost diacritics, out-of-lexicon words and Romanised regional-language/English code-mixing. The original assumptions of clean typed text, closed vocabulary, English-only morphology and exact accept/reject matching no longer hold.

New property	**Original component affected**	**Expected failure**
0/O, 1/l, rn/m substitutions	Regex and DFA validators	Identifiers, dates and drug names fail exact matching.
Line-break hyphenation	Tokenizer	One word becomes two fragments or an incorrect hyphenated token.
Lost diacritics	Lexicon and token matching	Regional words may not match stored forms.
≥20% OOV / misspelt / non-English tokens	FST analyser	Analysis coverage decreases.
Romanised Tamil-English mixing	English-only morphology and abbreviations	Tokens such as “ku”, “irundhuchu” and “kuduthen” remain unanalyzed.

If 20% of tokens are noisy or out of lexicon, exact-match analyser coverage has an upper bound near 80% before accounting for other lexicon gaps. Regex precision and recall also decrease because OCR substitutions can create false matches or break valid strings. Exact degradation must be measured on a labelled noisy sample.

B. Evaluation of the Original Design

Three example traces

OCR record: 'PT-I234567 05/II/2026 Dis-charge: fractured femur.'

Original pipeline: patient-ID regex fails because I is not 1; date regex may fail because II contains letters; tokenizer splits Dis-charge; FST may still analyse “fractured” if tokenised correctly.

Code-mixed record: 'patient ku fever irundhuchu, paracetamol 500mg kuduthen.'

Original pipeline: English tokenizer recognises patient, fever, paracetamol and 500mg; English FST does not analyse ku, irundhuchu or kuduthen. It cannot reliably infer the Tamil grammatical relations.

Mixed OCR/code-mixed record: 'pt O12345? fever irun-dhuchu; paracetarnol 5OOmg kuduthen.'

Original pipeline: identifier fails; line-break/inserted hyphen can split the regional token; paracetarnol is out of lexicon; 5OOmg contains letter O and fails numeric strength matching. This record requires confidence-aware correction, not unconditional regex replacement.

Evaluation method: annotate at least 3 noisy records initially, then expand to a representative noisy test set. Measure pattern precision/recall, FST coverage and retrieval F1 against the clean-corpus baseline. The exact numbers cannot be derived from the question sheet; they require execution on the noisy sample.

Formalism comparison

- Weighted finite-state transducers can assign costs to likely OCR corrections and preserve alternatives.
- Edit-distance matching can propose corrections such as O→0 or rn→m, but must be constrained by entity type and clinical lexicon to avoid unsafe substitutions.
- Character n-gram language identification can distinguish likely English, Romanised Tamil and unknown tokens at token or phrase level.

C. Adaptation / Redesign

Decision: MODIFY the original design rather than replace it. Keep deterministic finite-state processing for clean structured data, but add a noise-tolerant front end and language-aware fallback.

New pipeline: Raw OCR / typed notes → OCR cleanup + line-wrap repair → weighted edit-distance candidate generator → token-level language identification → regex/DFA entity recognition with confidence scores → English FST + regional-language lexicon/transducer → conservative normalisation → search index retaining original and corrected forms

Component	Change
Patterns	Add tolerant variants for common OCR confusions only in suitable fields; retain strict

	patterns for final validation.
Automata	Add weighted transitions/candidate scoring for noisy input; keep strict DFA validation after correction.
Lexicon	Expand with regional-language stems, Romanised variants, common clinical terms and OCR variants; mark language and confidence.
Rule cascade	Add line-break repair, diacritic normalisation and language-specific orthographic rules; do not force English rules onto Tamil tokens.
Search	Index original token, normalised token, lemma and correction candidates separately; rank lower-confidence matches below exact matches.

Expected effect: coverage should increase for noisy and code-mixed tokens, but precision may fall if correction is too aggressive. Runtime and memory increase because multiple correction candidates are considered. Annotation effort also increases because bilingual reviewers must validate Romanised forms and correction candidates. A confidence threshold and audit trail are necessary.

D. Evidence-Based Defence

The modified hybrid design is justified because strict regex and unweighted automata are efficient and reliable on clean structured fields but cannot tolerate common OCR errors or model regional-language morphology. A weighted correction layer addresses character noise; token-level language identification routes tokens to the appropriate analyser; and the original FST remains responsible for known English morphology. The Porter stemmer is retained only as a fallback, not as the primary clinical analyser.

The decision should be confirmed by comparing clean and noisy test sets using the same 40 queries and relevance judgements. Report the change in precision, recall, F1, analyser coverage and processing time. The remaining limitations are semantic ambiguity, negation and experiencer detection, unseen slang, context-dependent abbreviation meaning and arbitrary nested structures. No finite-state layer, however large, can by itself understand all clinical meaning or guarantee safe interpretation.

Conclusion

A morphology-aware finite-state layer provides a transparent foundation for hospital-record search. Regular expressions and automata recognise structured clinical entities, tokenisation and normalisation prepare text, and finite-state morphology connects inflected surface forms with stems and grammatical features. Porter stemming offers a fast fallback but may over-stem or under-stem clinical vocabulary. The recommended deployment is a hybrid design that preserves original text, uses conservative finite-state analysis for known forms, and adds weighted correction and language identification for OCR and code-mixed input. All numerical evaluation claims must be replaced with measured results from the documented synthetic or approved de-identified corpus before final submission.